Homework 6 Problem 1: Linear / Nonlinear Classification

Each subproblem in Problems 1 and 2 is worth 7 pts. Problems 3 and 4 are worth 10 and 11 points each. All cells are marked with instructions to insert your code. Please complete all cells as directed.

What to turn in:

- Please print the notebook containing the answers and results into a pdf file (you can use `File – Print`). Submit this pdf file to the main homework entry in gradescope. Be sure to locate your answers for each problem when you submit, as ususal. In the worst case where you cannot print it into a pdf file somehow, you can create a Microsoft word document and then copy-paste screenshots showing your code and output parts by parts.
- You also need to submit this jupyter notebook file filled with your answers in the code entry in gradescope.

Description: This homework will study 3-class classification in the famous "Iris" dataset. The dataset was introduced by the British statistician and biologist Ronald Fisher in his 1936 paper "The use of multiple measurements in taxonomic problems" as an example of linear discriminant analysis. The data set consists of 50 samples from each of three species of Iris (Iris setosa, Iris virginica and Iris versicolor). Four features were measured from each sample: the length and the width of the sepals and petals, in centimeters. Based on the combination of these four features, Fisher developed a linear classifier to distinguish the species from each other. We will do the same using classifiers that we have learned.

In [14]:
```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression, LogisticRegressionCV
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score
import warnings

# Suppress warnings
warnings.filterwarnings("ignore")
np.random.seed(0)
```

Problem 1 : Load / Explore Data

(a)

Because this dataset is so well-known, Scikit-Learn includes a special function for loading it, which is provided below. Do the following in the cell below:

- Load the data and create a Train / Test split with 25% test data (train_test_split() function)
- For the above, make sure to use the provided random state so that results are repeatable
- Display the training inputs (you can use function display())

Note: You will need the feature names later on. It is helpful at this point to store them in a set using the DataFrame.colums property.

In [15]:
```
# use this random state for train/test split
random_state=1234

iris = datasets.load_iris(as_frame=True)
X = iris.data
y = iris.target

# Insert code here
xTrain, xTest, yTrain, yTest = train_test_split(X, y, test_size=0.25, random

features = set(X.columns)
display(xTrain)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
22	4.6	3.6	1.0	0.2
93	5.0	2.3	3.3	1.0
36	5.5	3.5	1.3	0.2
68	6.2	2.2	4.5	1.5
32	5.2	4.1	1.5	0.1
...	...	...	...	...
143	6.8	3.2	5.9	2.3
116	6.5	3.0	5.5	1.8
53	5.5	2.3	4.0	1.3
38	4.4	3.0	1.3	0.2
47	4.6	3.2	1.4	0.2

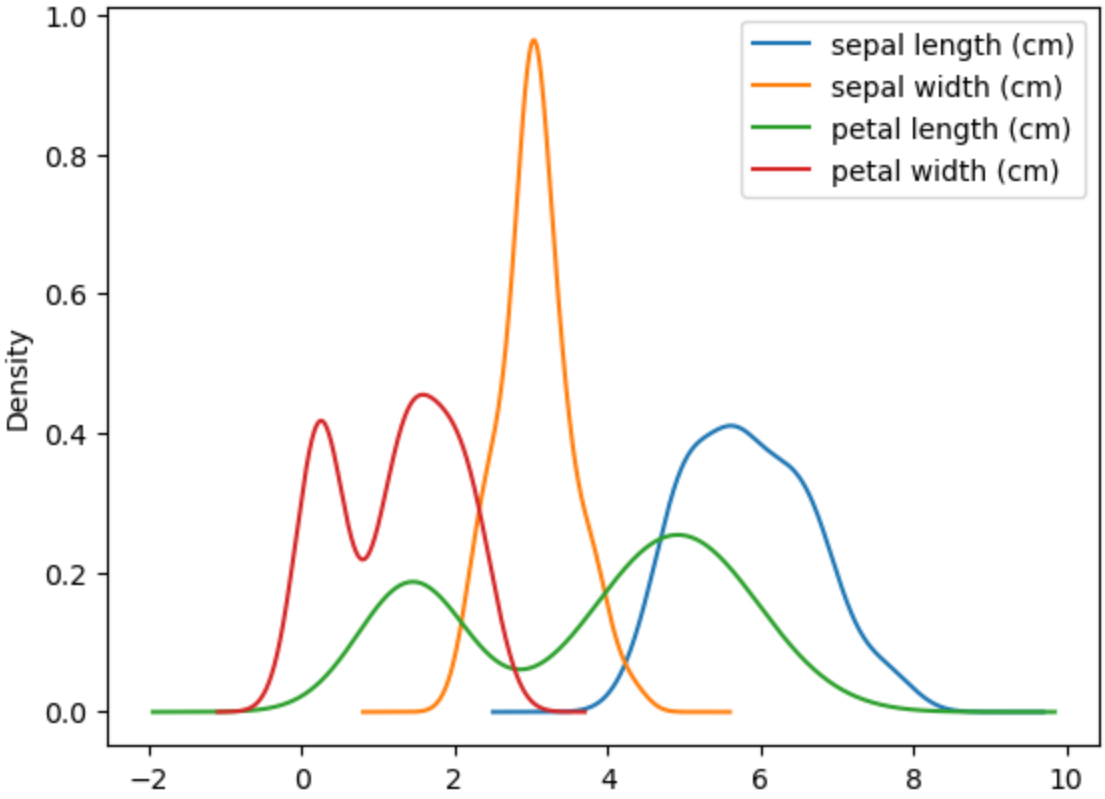
112 rows × 4 columns

(b)

Now we will explore our feature distributions. In the cell below, use the Pandas DataFrame plot feature to plot the density of each feature in the training data.

Documentation - Pandas - DataFrame.plot

In [16]:
```
xTrain.plot(kind="density")
plt.show()
```

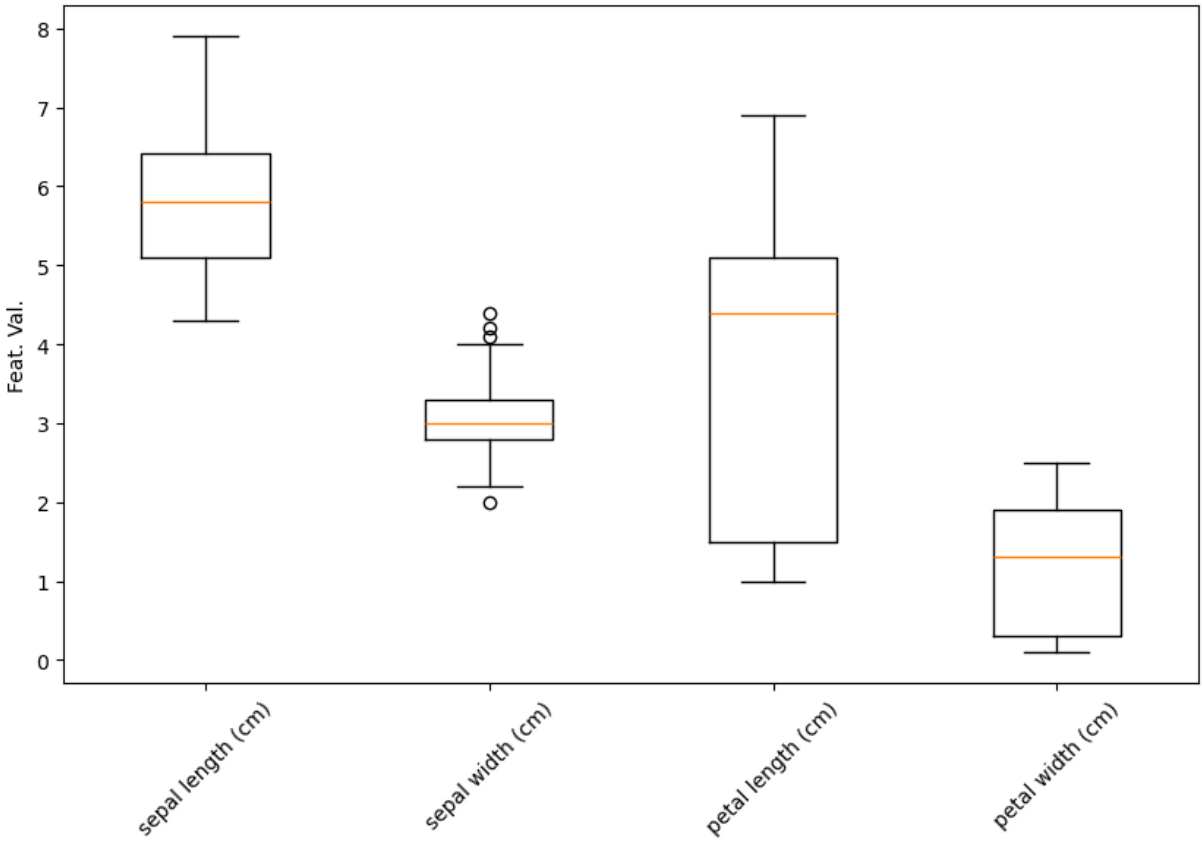


(c)

Sometimes it is better to look at distributions of each feature plotted together. In the cell below produce a boxplot (use plt.boxplot()) of each feature in the training data. **Make sure to rotate X-tick labels 45-degrees so they are readable.**

In [17]:
```
plt.figure(figsize=(10,6))

plt.boxplot([xTrain[col] for col in xTrain.columns], labels=xTrain.columns)
plt.xticks(rotation=45)
plt.ylabel("Feat. Val.")
plt.show()
```



(d)

Now let's see how well we can separate classes from each pair of features. In the cell below produce a scatterplot of **every pair of features** in the training data. There will be 6 scatterplots in all. Make sure to follow these instructions:

- Color each marker red, green, or blue depending on the true class label
- Use numpy.corrcoef to compute the correlation coefficient of each feature
- Title each plot with the correlation coefficient
- Label each axis using the corresponding feature name

In [19]:
```
# Insert code here
plt.figure(figsize=(10,10))

cols = list(xTrain.columns)
colors = ["red", "green", "blue"]
count = 1

for i in range(len(cols)):
    for j in range(i + 1, len(cols)):
        plt.subplot(3, 2, count)

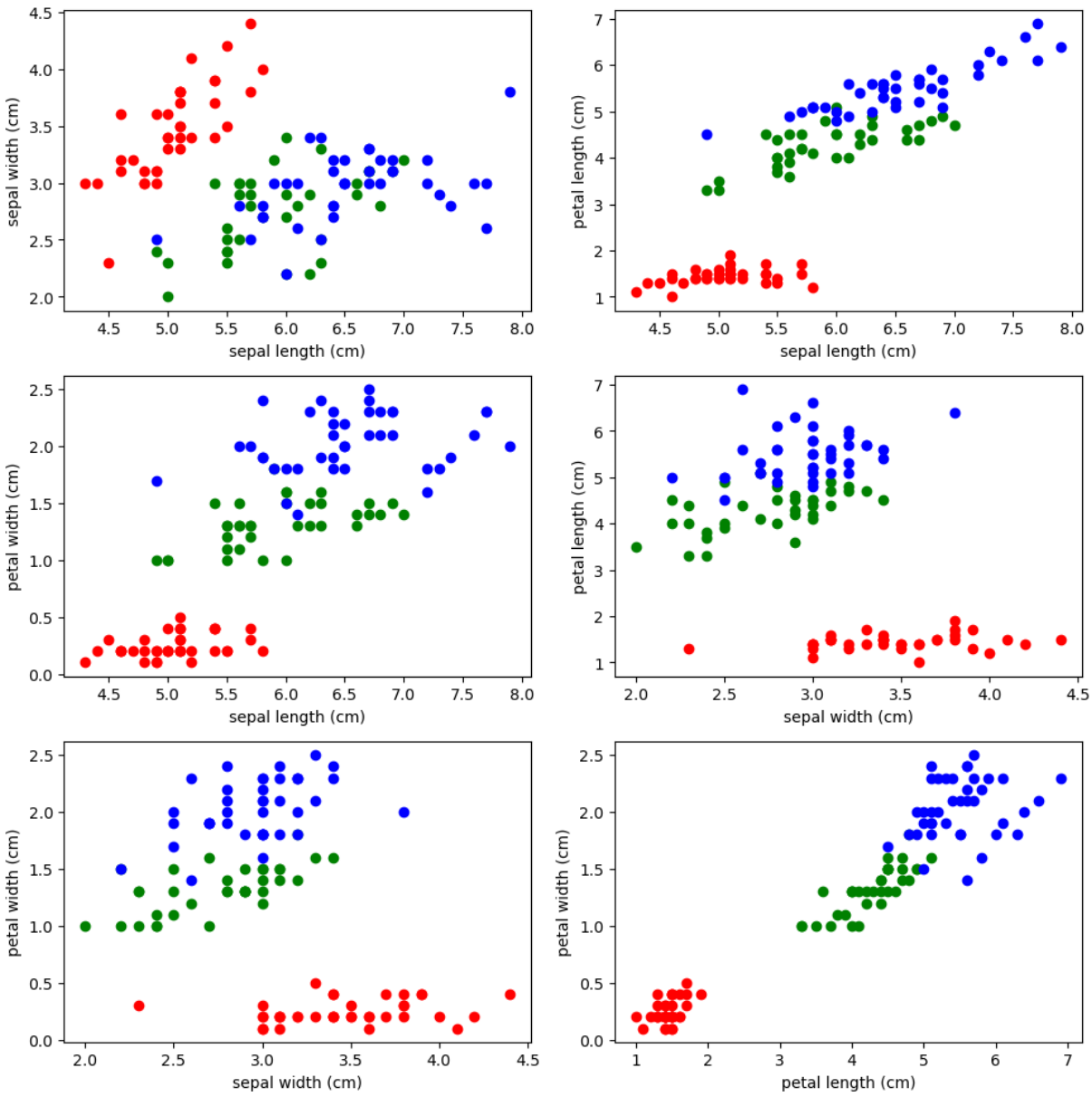
        x = xTrain[cols[i]]
        y = xTrain[cols[j]]
        corr = np.corrcoef(x, y)[0, 1]

        for k in range(3):
```

```
            idx = (yTrain == k)
            plt.scatter(x[idx], y[idx], color=colors[k])

        plt.xlabel(cols[i])
        plt.ylabel(cols[j])

        count += 1
plt.tight_layout()
plt.show()
```



Problem 2 : Train a logistic regression classifier

(a)

Now we will look at finding the best feature out of all the features. To do this, you will preform Cross Validation of Logustic Regression. We will break this into subproblems to

walk through it. In the cell do the following:

- Using LogisticRegressionCV perform 5-fold cross validation to train on each feature
- For each run use Matplotlib errorbar() to plot the average +/- standard deviation of error versus regularization coefficient (the property LogisticRegressionCV.Cs_) -- there should be 4 plots in total
- Set plot X-label with the feature name, and Y-label "Accuracy"
- Title each plot with the maximum achieved accuracy score
- Report the best accuracy from cross-validation
- Finally, report the best performing feature and save it for later

Make sure to set the following properties in LogisticRegressionCV:

- cv=5
- max_iter=1e4
- random_state=0
- multi_class='multinomial'
- Cs=10

Documentation - Scikit-Learn - LogisticRegressionCV

In [23]:
```
# Insert code here
plt.figure(figsize=(10,10))

cols = list(xTrain.columns)

topScore = -1
topFeat = ""

for i in range(len(cols)):
    feat = cols[i]
    oneFeat = xTrain[[feat]]

    model = LogisticRegressionCV(cv=5, max_iter=int(1e4), random_state=0, Cs
    model.fit(oneFeat, yTrain)

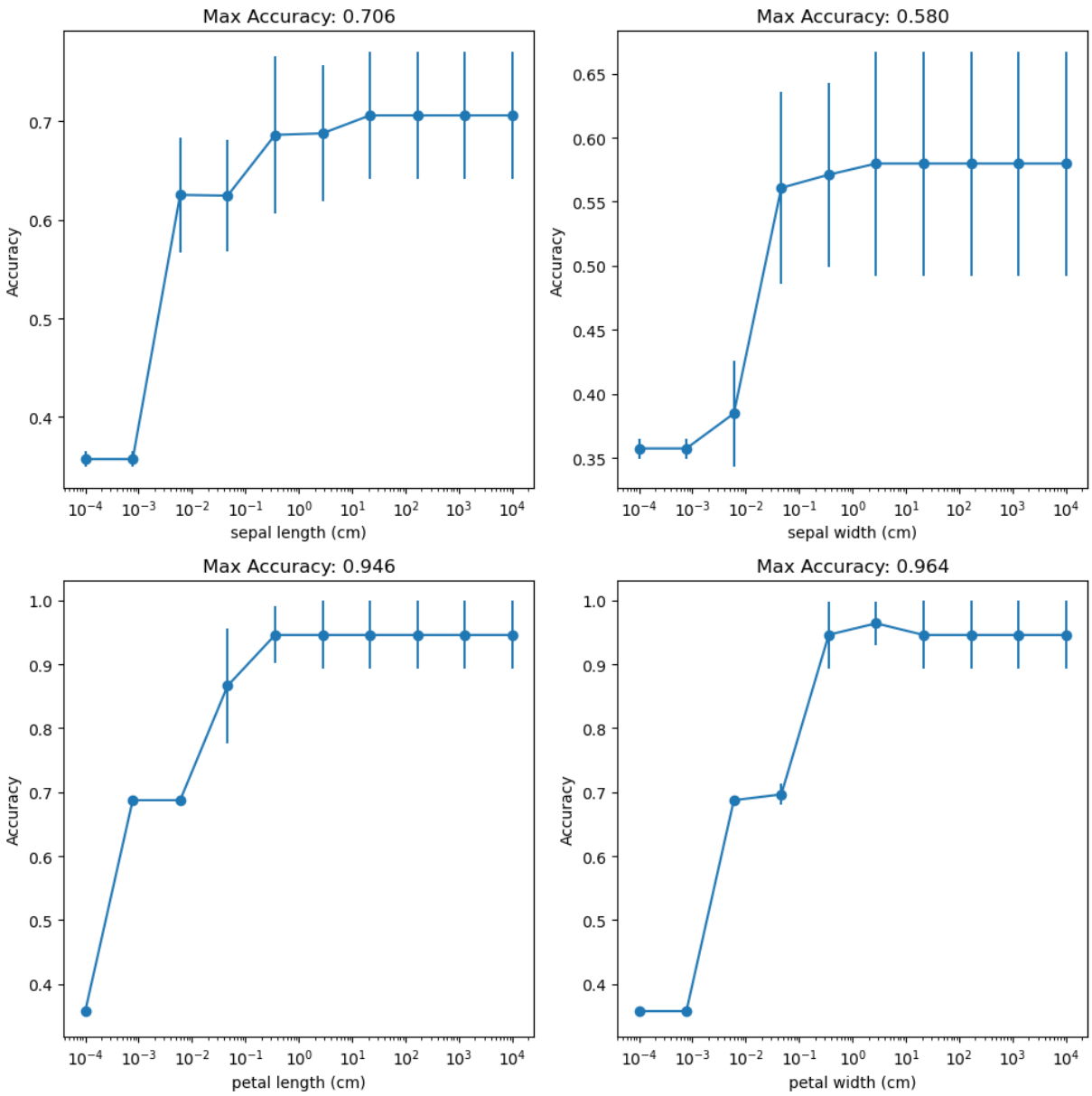
    meanScore = np.mean(model.scores_[0], axis=0)
    stdScore = np.std(model.scores_[0], axis=0)

    bestScore = np.max(meanScore)

    if bestScore > topScore:
        topScore = bestScore
        topFeat = feat

    plt.subplot(2, 2, i+1)
    plt.errorbar(model.Cs_, meanScore, yerr=stdScore, fmt='o-')
    plt.xscale('log')
    plt.xlabel(feat)
    plt.ylabel("Accuracy")
    plt.title(f"Max Accuracy: {bestScore:.3f}")
```

```
plt.tight_layout()
plt.show()
```



(b)

Now lets look at all pairs of features. The cell below provides a function plotLogreg2feat() to visualize the learned classifier for a pair of features. This function will draw the decision boundaries for each of the three classes, which will give us a better picture of what's going on. In the cell below that do the following:

- Loop over every pair of features (there are 6 pairs total)
- Using LogisticRegressionCV perform 5-fold cross validation to train a classifier on the pair of features
- Make sure to use **the same cross validation options as the previous experiment**
- Using plotLogreg2feat plot the learned classifier
- Title each plot with the maximum average accuracy from cross validation

In [22]:
```
def plotLogreg2feat(X, featname_1, featname_2, model):
    '''
    INPUTS:
      X - Input DataFrame (assumes Nx2 for N data points and 2 features)
      featname_1, featname_2 - String containing feature names
      model - Fitted LogisticRegressionCV model

    OUTPUTS:
      ax - Returns figure axis object
    '''

    # make grid
    x_min, x_max = X[featname_1].min() - 0.5*X[featname_1].std(), X[featname
    y_min, y_max = X[featname_2].min() - 0.5*X[featname_2].std(), X[featname
    h = 0.02  # step size in the mesh
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max,
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])

    # Put the result into a color plot
    Z = Z.reshape(xx.shape)
    #plt.figure(1, figsize=(4, 3))
    fig, ax = plt.subplots()
    ax.pcolormesh(xx, yy, Z, cmap=plt.cm.Paired)

    # Plot also the training points
    #plt.scatter(X_train[features[i]][ Y_train == 0 ], X_train[features[j]][
    #plt.scatter(X_train[features[i]][ Y_train == 1 ], X_train[features[j]][
    #plt.scatter(X_train[features[i]][ Y_train == 2 ], X_train[features[j]][

    ax.scatter(X[featname_1][ y == 0 ], X[featname_2][ y == 0 ], c='r')
    ax.scatter(X[featname_1][ y == 1 ], X[featname_2][ y == 1 ], c='g')
    ax.scatter(X[featname_1][ y == 2 ], X[featname_2][ y == 2 ], c='b')

    ax.set_xlabel(featname_1)
    ax.set_ylabel(featname_2)
```

```
    #plt.xlim(xx.min(), xx.max())
    #plt.ylim(yy.min(), yy.max())
    ax.set_xticks(())
    ax.set_yticks(())
    return ax
```

In [26]:
```
# Insert code here

cols = list(xTrain.columns)

pairtopScore = -1
pairtopFeat = ()

for i in range(len(cols)):
    for j in range(i + 1, len(cols)):
        feat1 = cols[i]
        feat2 = cols[j]

        twoFeat = xTrain[[feat1, feat2]]

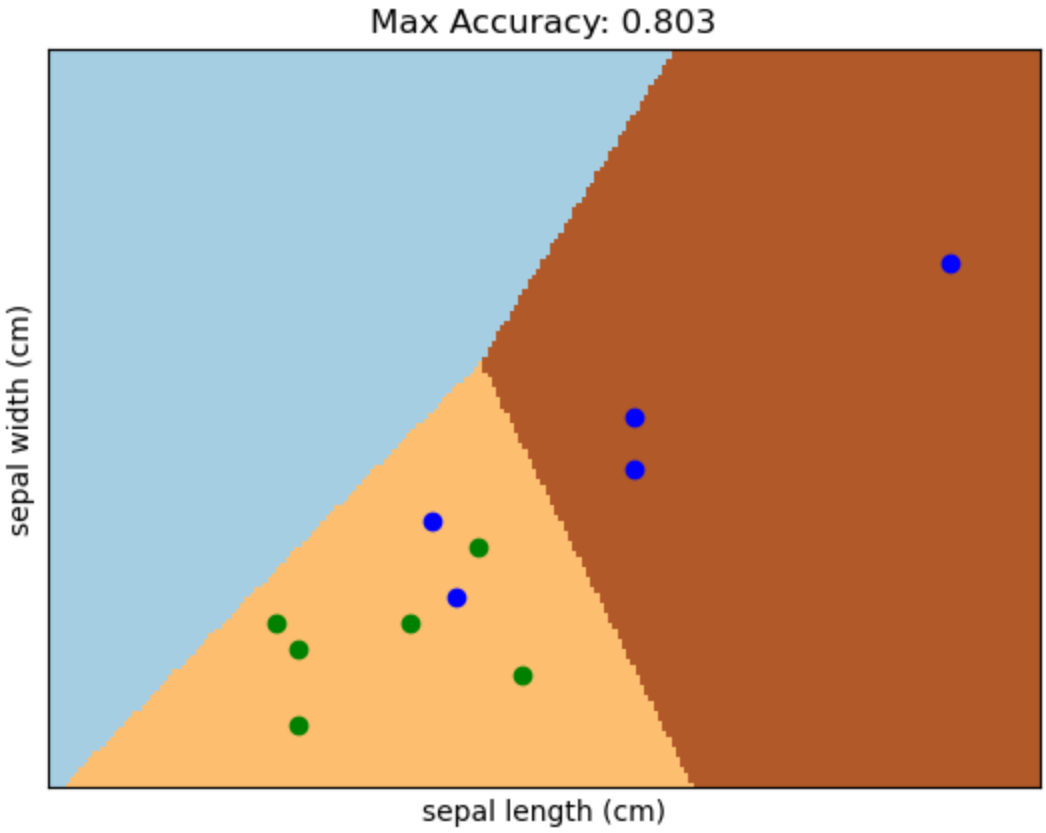
        model = LogisticRegressionCV(cv=5, max_iter=int(1e4), random_state=0
        model.fit(twoFeat, yTrain)

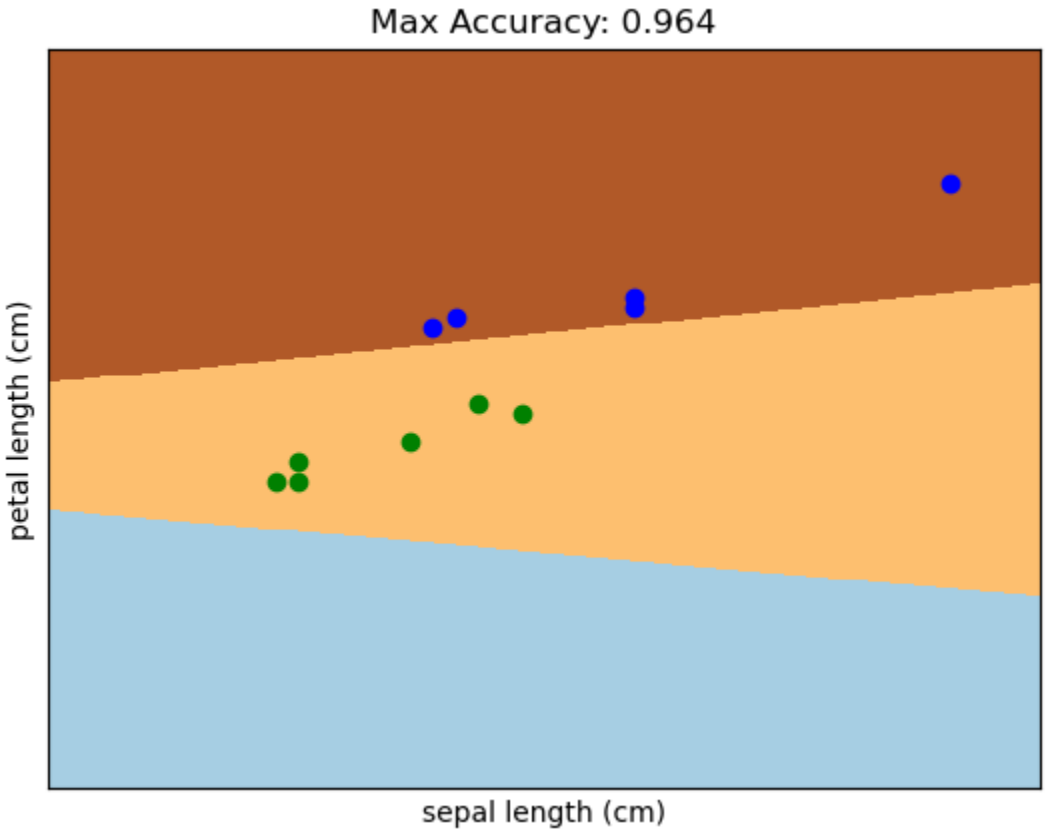
        scorings = list(model.scores_.values())[0]
        meanScore = np.mean(scorings, axis=0)
        bestScore = np.max(meanScore)

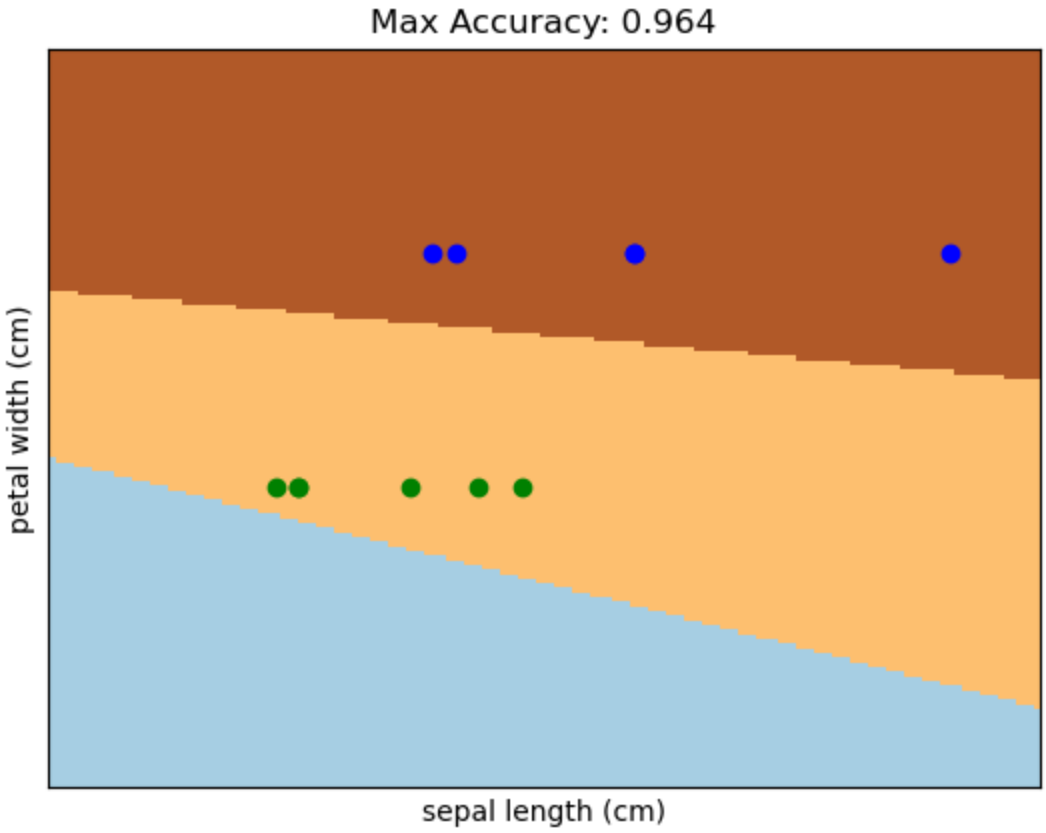
        if bestScore > pairtopScore:
            pairtopScore = bestScore
            pairtopFeat = (feat1, feat2)

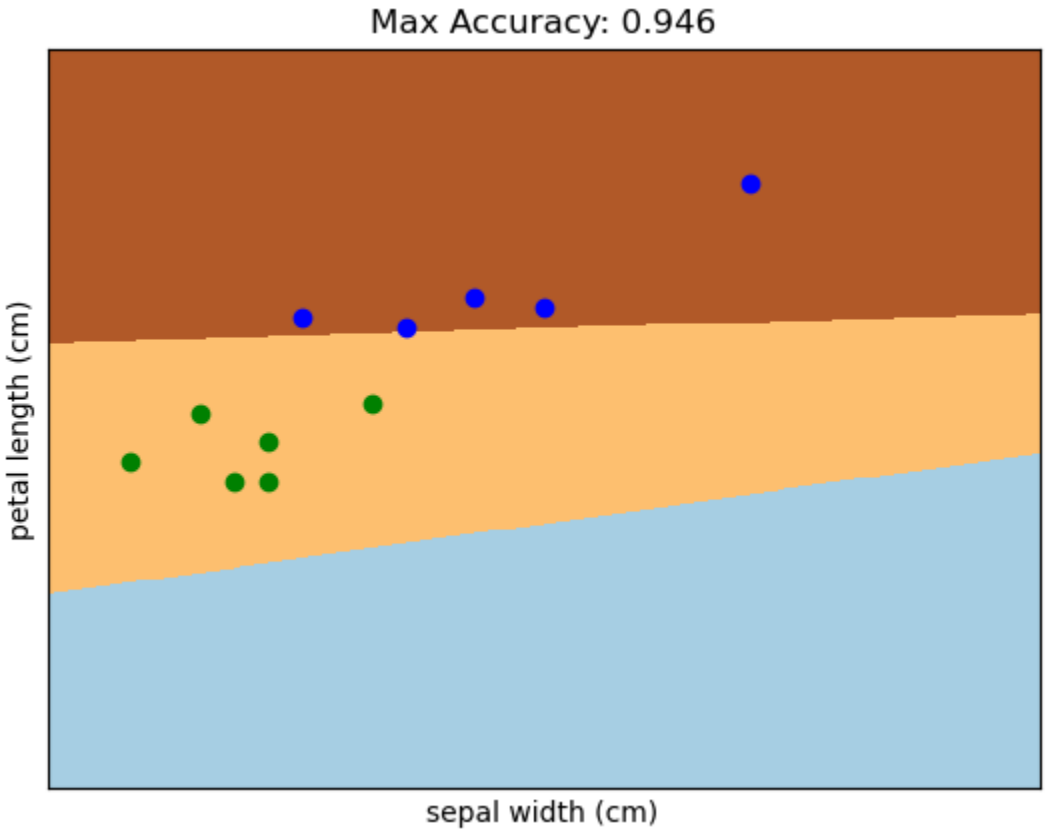
        ax = plotLogreg2feat(twoFeat, feat1, feat2, model)
        ax.set_title(f"Max Accuracy: {bestScore:.3f}")
plt.show()
```

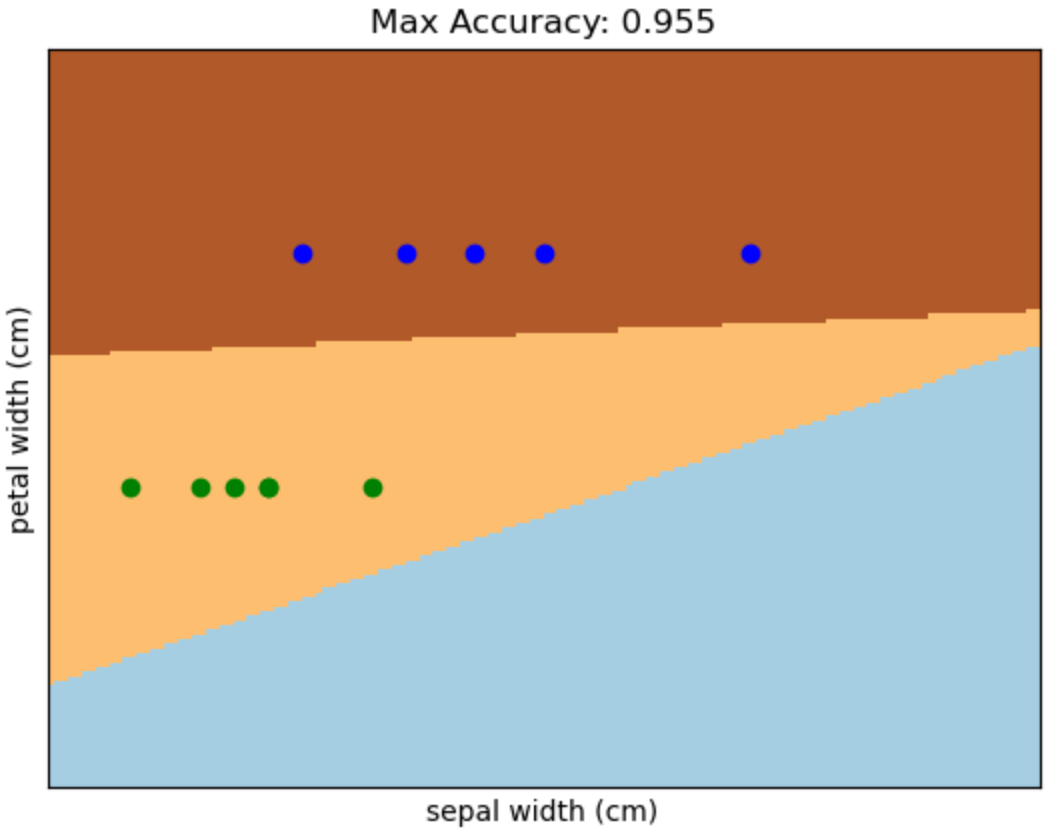
```
<Figure size 1000x1000 with 0 Axes>
```

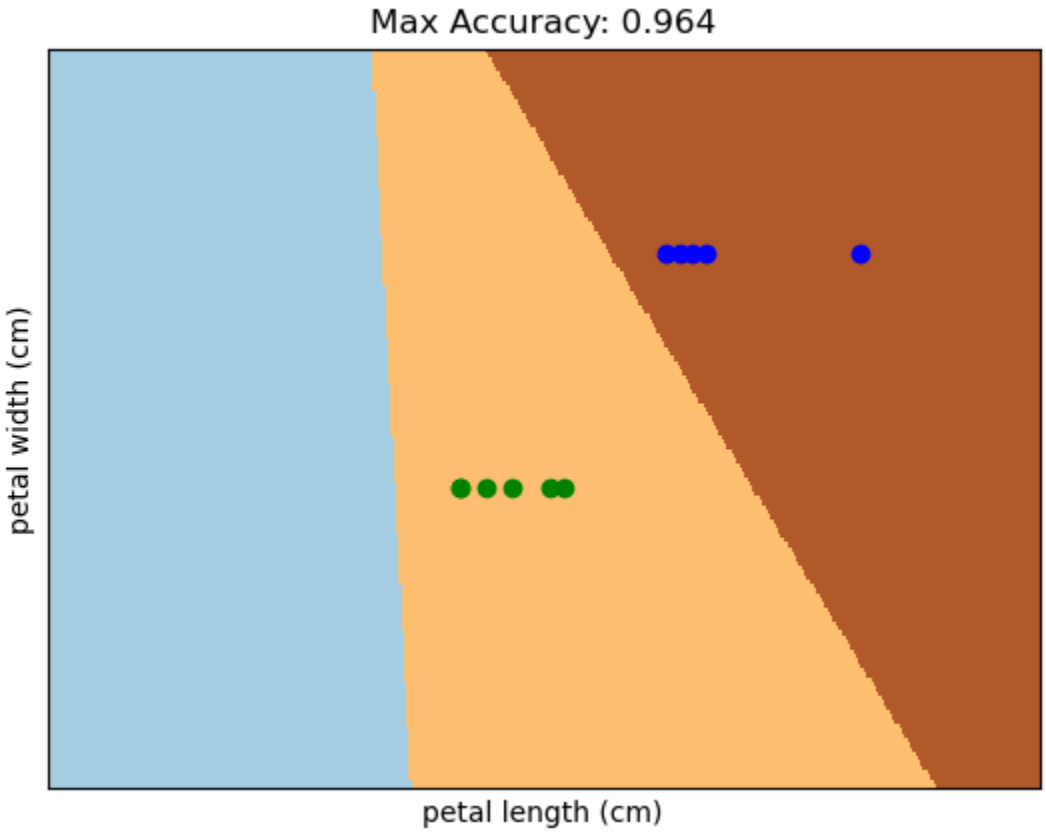

Max Accuracy: 0.803
sepal width (cm)
sepal length (cm)


Max Accuracy: 0.964
petal length (cm)
sepal length (cm)


Max Accuracy: 0.964
petal width (cm)
sepal length (cm)


Max Accuracy: 0.946
petal length (cm)
sepal width (cm)





(c)

Surprisingly, adding pairs of features doesn't seem to improve things. Let's try training

on all features. In the cell below:

- Perform 5-fold cross validation (using all the same parameters as before) to train a logistic regression classifier on all features
- Report the maximum of the average scores.

In [34]:
```
# Insert code here
model = LogisticRegressionCV(cv=5, max_iter=int(1e4), random_state=0, Cs=10)
model.fit(xTrain, yTrain)
scorings = list(model.scores_.values())[0]
meanScore = np.mean(scorings, axis=0)
bestScore = np.max(meanScore)

print(f"Max average score: {bestScore:.3f}")
```

```
Max average score: 0.964
```

If your results are the same as mine, the maximum score over all features is the same as over the best single feature.

Problem 3 : Support Vector Machine

We have trained several logistic regression classifiers, all of which achieve an a cross-validation accuracy well into the 90% range. In an effort to see if we can do better, let's train one lass classifier--a support vector machine. For this classifier we will introduce a nonlinear tranformation using the Radial Basis kernel function. In the cell be low do the following:

- Using Numpy.logspace create a logarithmically spaced set of 100 regularization coefficient in the range (1e-4, 1e4)
- For each coefficient define a support vector classifier with kernel='rbf' and set the regularization coefficient (C=coefficient)
- Perform 5-fold cross validation (e.g. using cross_val_score)
- Plot the average accuracy versus regularization coefficient and report the maximum accuracy and best coefficient
- Make sure to set the plot X-scale to 'log' and label axes and title

Documentation - Scikit-Learn - svm.SVC

In [33]:
```
# Insert code here
coefficients = np.logspace(-4, 4, 100)
averageScores = []
topScore = -1
topCoefficient = 0

for coef in coefficients:
    model = SVC(kernel='rbf', C=coef)
    scores = cross_val_score(model, xTrain, yTrain, cv=5)
    averageScore = np.mean(scores)
```

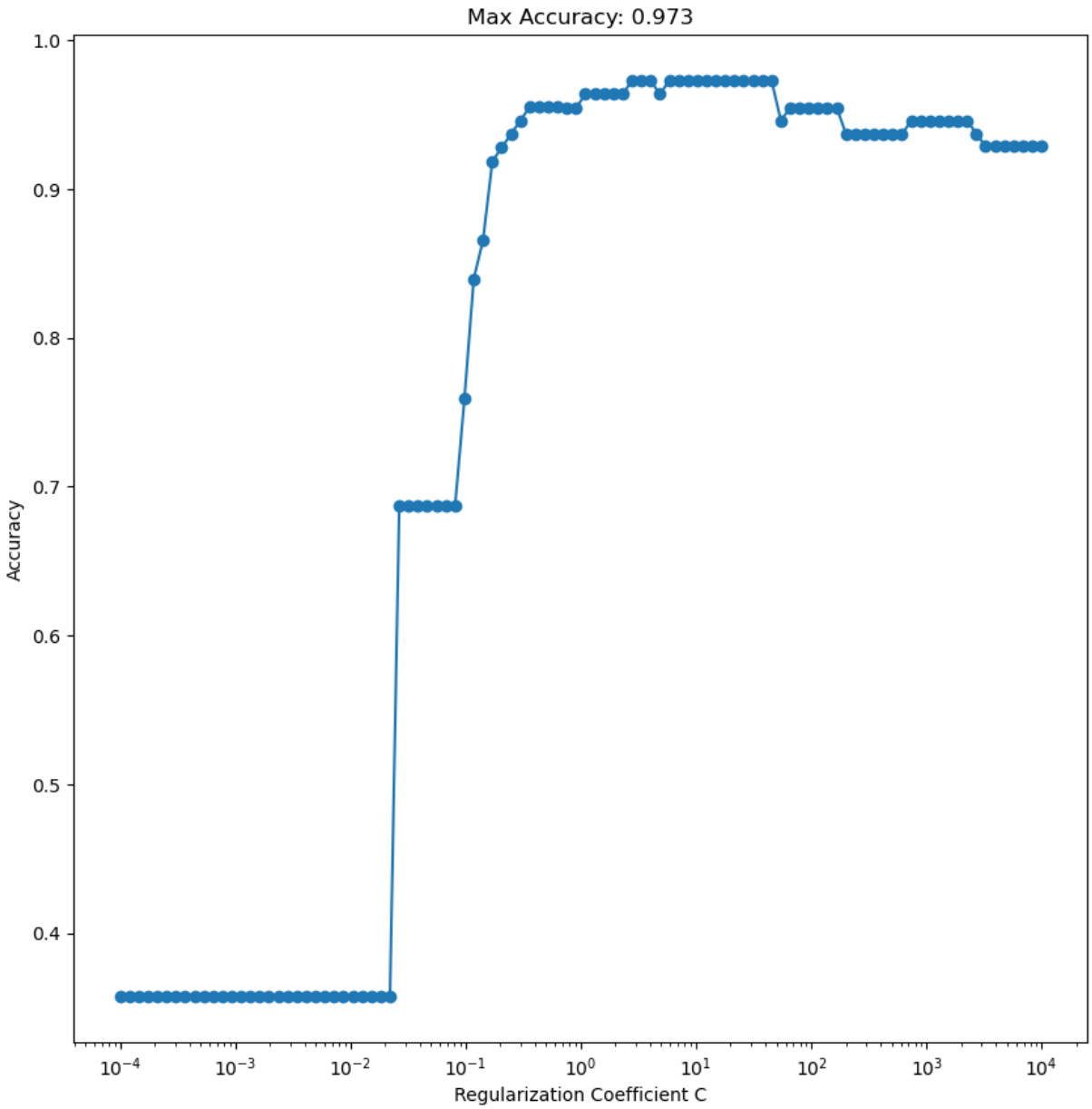
```
    averageScores.append(averageScore)

    if averageScore > topScore:
        topScore = averageScore
        topCoefficient = coef

plt.figure(figsize=(10,10))

plt.plot(coefficients, averageScores, 'o-')
plt.xscale('log')
plt.xlabel("Regularization Coefficient C")
plt.ylabel("Accuracy")
plt.title(f"Max Accuracy: {topScore:.3f}")
plt.show()

print("Best Coefficient: ", topCoefficient)
print("Max Accuracy: ", topScore)
```



```
Best Coefficient:  2.782559402207126
Max Accuracy:  0.9727272727272727
```

My results show slightly higher accuracy under the SVM classifier. However, cross_val_score does not use the same cross validation splits as the built-in cross validation of LogisticRegressionCV (which randomizes splits). So we shall see...

Problem 4 : Evaluate on test

Now we will evaluate all classifiers on the test data. Take the best regression classifier and the best SVM classifier (with previously chosen parameters), train them, and evaluate accordingly. For each classifier report:

- Test accuracy
- Confusion matrix
- Results of classification_report

We have only left a single cell below. Feel free to insert additional cells and arrange output as you see fit. Make sure it is readable. Feel free to explore any additional visualizations or metrics.

In [36]:
```
# This is for Logistic Regression
bestLogisticRegression = LogisticRegressionCV(cv=5, max_iter=int(1e4), rando
bestLogisticRegression.fit(xTrain, yTrain)
logPrediction = bestLogisticRegression.predict(xTest)
print("Title: Logistic Regression")
print("Test Accuracy:", bestLogisticRegression.score(xTest, yTest))

classes = np.unique(yTest)
confusionMatrix = np.zeros((len(classes), len(classes)), dtype=int)
for i in range(len(yTest)):
    actual = yTest.iloc[i]
    prediction = logPrediction[i]
    confusionMatrix[actual, prediction] += 1

print("\nConfusion Matrix:")
print(confusionMatrix)

print("\nClassification Report:")
for c in classes:
    tp = confusionMatrix[c, c]
    fp = np.sum(confusionMatrix[:, c]) - tp
    fn = np.sum(confusionMatrix[c, :]) - tp
    support = np.sum(confusionMatrix[c, :])
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recal

    print(f"Class {c}:")
    print(f"Precision: {precision:.3f}")
    print(f"Recall: {recall:.3f}")
```

```
    print(f"F1-score: {f1:.3f}")
```

```
Title: Logistic Regression
Test Accuracy: 0.9736842105263158

Confusion Matrix:
[[13  0  0]
 [ 0 14  1]
 [ 0  0 10]]

Classification Report:
Class 0:
Precision: 1.000
Recall: 1.000
F1-score: 1.000
Class 1:
Precision: 1.000
Recall: 0.933
F1-score: 0.966
Class 2:
Precision: 0.909
Recall: 1.000
F1-score: 0.952
```

In [ ]: